Name- Chirag Saxena

Scholar No. 2211301212

Electrical Department

DSA Assignment 03

1.) Write the implementation procedure of basic primitive operations of the Queueusing:(i)Lineararray. (ii)linkedlist.

sol.)

1. Array-Based Queue Implementation

In an array-based queue, two variables, front and rear, track the position of the first and last elements in the queue.

(i) Enqueue Operation (Add Element):

- CheckforOverflow:Ensurethequeueisnotfullbycheckingifrear==size-1.
- AddElement:Insertthenewelementatthepositionpointedtobytherear.
- UpdateRear:Incrementthereartothenextposition.

(ii) Dequeue Operation (Remove Element):

- CheckforUnderflow:Ensurethequeueisnotemptybycheckingiffront>rear.
- RemoveElement:Retrievetheelementatthepositionpointedtobythefront.
- UpdateFront:Incrementthefronttothenextposition.

2. Linked List-Based Queue Implementation

In a linked list-based queue, each node contains data and a pointer to the next node. Two pointers, front (for dequeue) and rear (for enqueue), are used to track the ends of the queue. This allows the queue to dynamically grow and shrink without a fixed size limitation.

(i) Enqueue Operation (Add Element):

- CreateNewNode:Allocatememoryforanewnodewiththedatatobeadded.
- Link Node: If the queue is empty, both front and rear point to the new node. Otherwise, link the current rear node to the new node.
- UpdateRear:Setreartopointtothenewnode.

(ii) Dequeue Operation (Remove Element):

- CheckforUnderflow:Ensurethequeueisnotempty(i.e.,front!=NULL).
- RemoveElement:Retrievethedataatthefrontnode.
- UpdateFront:Movethefrontpointertothenextnode.

```
 cout<<”Enter the value you want to add to DLL:”;
cin>>value;
    node* head= new node (value);
    node* prev=head;
    for(int i=1;i<n;i++){
          cout<<”Enter the value you want to add to DLL:”;
          cin>>value;
        node* temp= new node(value,nullptr,prev);
        prev->next=temp;
        prev=temp;
    }
    return head;

void PrintDLL(node* head){
    node*temp= head;
    while(temp!=nullptr){
        cout<<temp->data<<"-->";
        temp=temp->next;
```

ii) For a given singly linked list delete a node:

a. at the beginning

```
void DeleteHead(node* head){
if(head==NULL) return;
    node* temp =head;
    head=head->next;
    temp->next=NULL;
    delete temp;
```

2.) Write a pseudo code for following:

(i)Take N numbers as input from the user and create a doubly linked list.

sol.)

```
#include<bits/stdc++.h>
using namespace std;
class node{
public:
int data;
node* next;
node* back;


   public:
   node(int x, node* next1, node* back1){
      data=x;
      next=next1;
      back=back1;
   }


   public:
   node(int x){
      data= x;
   }
};
int n;
cout<<"Enter the number Of elements you want to add:";
cin>>n;


node* convertToDLL(int n){
  int value;
```

b.at the end.

```
 void DeleteEnd(node* head){
if(head==NULL) return;
if(head->next== NULL){
    Delete head;
    Return;}
node* prev;
  node* temp =head->next;
 while(temp->next != NULL){
          prev=temp;
           temp=temp->next;}
prev->next=NULL;
delete temp;}
```

c. at a given position k.

Input: k = 3

```
 void DeleteKth(node* head, int n){
if(head==NULL) return;
if(head->next== NULL){
    Delete head;
    Return;}
node* prev;
  node* temp =head->next;
int i=0;
while(i<n){
          prev=temp;
           temp=temp->next; i++; }
```

```
prev->next=NULL;
delete temp;}
```

3.) What data structure is used in solving the Josephus problem? How to solve the Josephus problem?

sol.) The Josephus problem can be efficiently solved using a circular linked list. However, it can also be solved using an array or mathematical approaches, depending on the complexity and the method of solving. A circular linked list is the most intuitive because it allows easy traversal in a circular manner, mimicking the elimination process in the problem.

The problem is a theoretical problem related to a certain elimination game. Given a group of people standing in a circle, every k-th person is eliminated until only one person remains. The objective is to determine the position of the last person remaining.

```
#include <bits/stdc++.h>
using namespace std;
class node{
public:
int data;
node* next;
node(int x){
        data = x;
        next= NULL;
    }
    node(int x,node* address){
        data = x;
        next= address;
    }
};
int main(){
    int n,k,i,count;
```

```
node *start,*ptr,*deleter;
cout<<"Enter Number of Players:";
cin>>n;
cout<<"enter the elimination Number:";
cin>>k;
start= new node(1);
ptr=start;
for(i=1;i<n;i++){
    node* temp=new node(i+1);
    ptr->next=temp;
    ptr=ptr->next;}
    ptr->next=start;

for(count=n; count>1;count--){
    for(i=0;i<k-1;i++) ptr=ptr->next;
    deleter=ptr->next;
    ptr->next=ptr->next->next;
    delete deleter;
}
cout<<"\nThe winner of Josephus Problem is "<<ptr->data;
return 0;}
```